

学 号

2104230414



沈阳建筑大学

课程设计任务书

课程设计名称 数据结构课程设计

(2023 年春季学期)

学 院 计算机科学与工程学院

专 业 计算机科学与技术

班 级 计算机 2101 班

姓 名 张清晨

指导教师 孙焕良、刘俊岭

2023 年 6 月 1 日

数据结构课程设计任务书

设计时间	2023 年 7 月 3 日 — 2023 年 7 月 7 日	
设计题目	表达式转换	
设计条件	硬件：计算机。 软件：Windows 操作系统，Visual C++ 6.0。	
设计任务	(1) 以字符形式输入语法正确的常规表达式； (2) 根据表达式构造对应的二叉树； (3) 利用二叉树的非递归算法输出常规表达式对应的前缀表达式； (4) 利用二叉树的非递归算法输出常规表达式对应的后缀表达式；	
设计要求	(1) 程序设计说明完整； (2) 程序框图清晰； (3) 源程序代码可读性好； (4) 程序可执行； (5) 严禁雷同。	
进度计划	时间（日期）	设计内容
	2023.7.3	教师布置设计任务，学生选题。查阅资料，基本技术准备。
	2023.7.4	设计程序框图和功能。编写程序。
	2023.7.5	调试程序。
	2023.7.6	程序演示。
	2023.7.7	撰写设计说明书。提交报告，评定成绩。
成绩评定办法 （评定考核项目、考核标准、所占比重）	课设答辩 ：40% 演示说明课程设计要求实现的功能。 课程设计说明书 ：40% 书写工整，项目填写齐全，内容正确。 平时表现 ：20% 按时出勤，纪律良好。 注：最后一天为答辩，重点检查自行完成部分，有创新之处、程序质量高，现场编写。最终成绩按优、良、中、及格、不及格记载。	
参考资料	《数据结构题集（c 语言）》 严蔚敏,吴伟民， 北京：清华大学出版社。 《数据结构 (C 语言版)》 严蔚敏,吴伟民， 北京：清华大学出版社。	
指导教师（签字）	2023 年 6 月 1 日	
教研室主任（签字）	2023 年 6 月 1 日	

学号	2104230414	成绩	
----	------------	----	--



沈阳建筑大学

课程设计说明书

设计名称 数据结构课程设计

设计题目 表达式转换

设计时间 2023 年 7 月 3 日—7 月 7 日

学 院 计算机科学与工程学院

专 业 计算机科学与技术

班 级 计算机 2101 班

姓 名 张清晨

指导教师 孙焕良、刘俊岭

2023 年 7 月 7 日

数据结构课程设计说明书

目 录

- 一、 题目简介
- 二、 功能说明
- 三、 程序框图
- 四、 程序清单
- 五、 运行结果
- 六、 设计体会
- 七、 参考文献
- 八、 教师评语

一、题目简介

1 问题描述

写一个程序，根据输入的算术表达式构造对应的二叉树

2 基本要求：

- (1) 以字符形式输入语法正确的常规表达式；
- (2) 根据表达式构造对应的二叉树；
- (3) 利用二叉树的非递归算法输出常规表达式对应的前缀表达式；
- (4) 利用二叉树的非递归算法输出常规表达式对应的后缀表达式；

二、功能说明

设计了一个表达式运算的计算器系统。该计算器支持输入中缀表达式，并可以将中缀表达式转换为前缀和后缀形式。它还可以计算中缀、前缀和后缀形式的表达式。具有以下基本功能：

1. 输入中缀表达式

输入想要计算的(中缀)表达式：如 $1+2+2*3-4/5$

2. 中缀表达式求值

显示中缀表达式和运算结果

3. 前缀表达式求值

显示前缀表达式和运算结果

4. 后缀表达式求值

显示后缀表达式和运算结果

5. 表达式的各种形式

显示表达式的前缀、中缀、后缀表达式

6. 退出系统

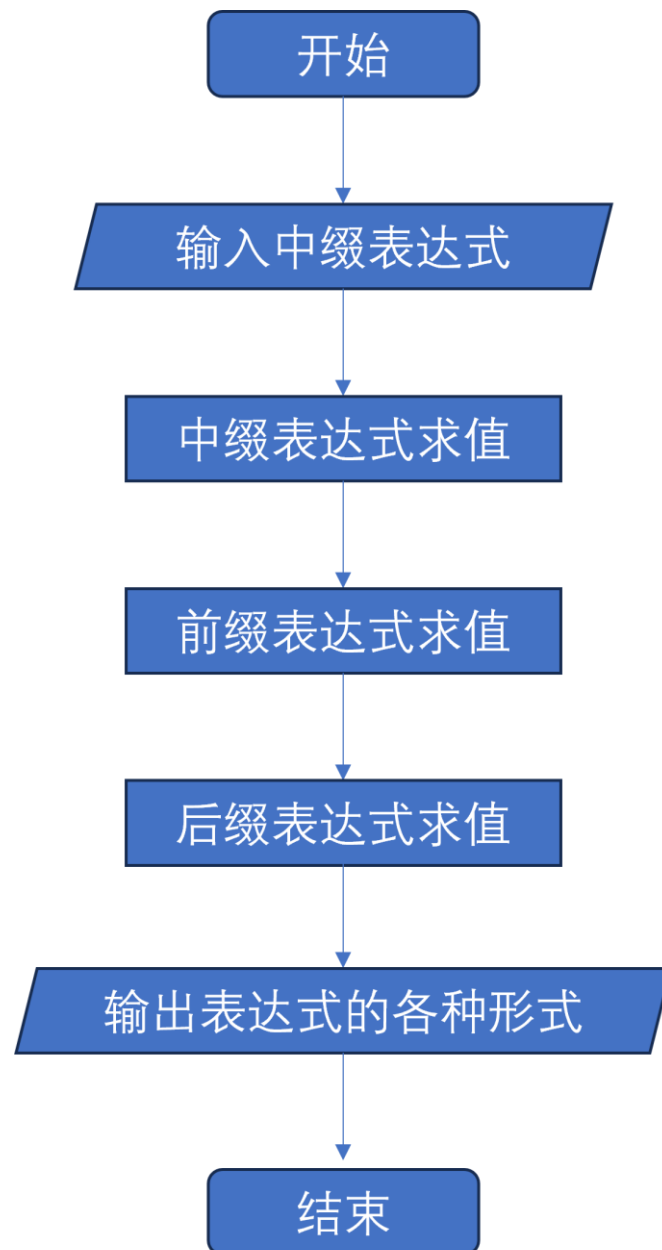
离开系统

该程序使用栈数据结构将中缀表达式转换为前缀和后缀形式。它定义了一个名为 "Caculate" 的类, 其中包含各种成员函数来处理计算器的不同操作。以下是该类及其成员函数的概述:

- 类: Caculate
 - 成员变量:
 - is_Input: 布尔变量, 用于跟踪是否已输入表达式。
 - infix_Expression: 存储中缀表达式的向量。
 - prefix_Expression: 存储前缀表达式的向量。
 - postfix_Expression: 存储后缀表达式的向量。
 - 成员函数:
 - show_Menu(): 显示菜单选项。
 - get_Value(): 返回运算符的优先级值。
 - get_result(): 计算简单表达式的结果。
 - input_Infix(): 允许用户输入中缀表达式, 并将其转换为前缀和后缀形式。
 - to_Prefix(): 将中缀表达式转换为前缀形式。
 - to_Postfix(): 将中缀表达式转换为后缀形式。
 - caculate_Prefix(): 计算前缀表达式的值。
 - caculate_Infix(): 计算中缀表达式的值。
 - caculate_Postfix(): 计算后缀表达式的值。
 - show_Vector(): 显示向量的元素。
 - show_Expression(): 显示中缀、前缀和后缀表达式。
 - exit_System(): 退出程序。

主函数包含一个循环, 重复显示菜单, 接受用户输入, 并根据用户的选择执行相应的操作。

三、程序框图



四、程序清单

```
#include<iostream>
#include<stack>
#include<string>
#include<sstream>//用来进行流的输入、输出和输入输出操作
#include<vector>//向量（Vector）是一个封装了动态大小数组的顺序容器
#include<algorithm>

using namespace std;

class Caculate {
public:
    //是否输入了表达式, 默认 false
    bool is_Input = false;
    //展示菜单
    void show_Menu();
    //获取运算符优先级
    int get_Value(string operation);
    //获取简单表达式计算结果
    //a 代表操作数 1, str 代表 操作运算符, b 代表操作数 2
    // a str b: 例如 a - b, a*b
    int get_result(int a, string str, int b);
    //输入中缀表达式
    void input_Infix();
    //存储中缀表达式
    vector<string> infix_Expression;
    //存储前缀表达式
    vector<string> prefix_Expression;
    //存储后缀表达式
    vector<string> postfix_Expression;
    //由中缀表达式转成前缀表达式
    void to_Prefix();
    //由中缀表达式转成后缀表达式
    void to_Postfix();
    //前缀表达式计算求值
    void caculate_Prefix();
    //中缀表达式计算求值
    void caculate_Infix();
    //后缀表达式计算求值
    void caculate_Postfix();
    //遍历展示 vector 容器中元素
```



```

void show_Vector(vector<string> expression);
//展示表达式的各种形式
void show_Expression();
//退出系统
void exit_System();
};

int main() {
    Caculate ca;
    int choice = 0;

    while (true) {
        ca.show_Menu();
        cout << "请输入你的选择: ";
        cin >> choice;
        switch (choice)
        {
            case 0:
                ca.exit_System(); //退出系统
                break;
            case 1:
                ca.input_Infix(); //输入中缀表达式
                break;
            case 2:
                ca.caculate_Infix(); //中缀表达式求值
                break;
            case 3:
                ca.caculate_Prefix(); //前缀表达式求值
                break;
            case 4:
                ca.caculate_Postfix(); //后缀表达式求值
                break;
            case 5:
                system("cls");
                ca.show_Expression(); //表达式的各种形式
                break;
            default:
                cout << "请重新输入你的选择: ";
                break;
        }
    }

    system("pause");
}

```

```

        return 0;
    }

```

//退出系统

```

void Caculate::exit_System() {
    cout << "欢迎下次使用！ :)" << endl;
    system("pause");
    exit(0);
}

```

//展示菜单

```

void Caculate::show_Menu() {
    cout << endl;
    cout << "*****" << endl;
    cout << "*****  表达式求值计算器  *****" << endl;
    cout << "*****" << endl;
    cout << "***** >1.  输入中缀表达式  *****" << endl;
    cout << "***** >2.  中缀表达式求值  *****" << endl;
    cout << "***** >3.  前缀表达式求值  *****" << endl;
    cout << "***** >4.  后缀表达式求值  *****" << endl;
    cout << "***** >5.  表达式的各种形式 *****" << endl;
    cout << "***** >0.  退出系统      *****" << endl;
    cout << "*****" << endl;
    cout << "*****" << endl;
    cout << endl;
}

```

//获取元素优先级

```

int Caculate::get_Value(string operation) {
    int value = 0;

    if (operation == "+") {
        value = 1;
    }
    else if (operation == "-") {
        value = 1;
    }
    else if (operation == "*") {
        value = 2;
    }
    else if (operation == "/") {
        value = 2;
    }
    else if (operation == "%") {

```

```

        value = 2;
    }
    else {
        value = 0;
    }

    return value;
}

```

//获取简单表达式计算结果

```

int Caculate::get_result(int a, string str, int b) {
    int result = 0;
    if (str == "+") {
        result = a + b;
    }
    else if (str == "-") {
        result = a - b;
    }
    else if (str == "*") {
        result = a * b;
    }
    else if (str == "/") {
        result = a / b;
    }
    else if (str == "%") {
        result = a % b;
    }
    else {
        result = 0;
    }

    return result;
}

```

//输入中缀表达式

```

void Caculate::input_Infix() {
    this->infix_Expression.clear();          //清空容器，方便接收用户接下来的输入
    system("cls");

    cout << "请输入你想要计算的(中缀)表达式: " << endl;
    char cha[50];                            //用来接收将要输入的表达式，最多接收 50 个字符
    cin.ignore();                             //cin.getline 默认以换行符为结束标志,要消耗掉前面的换行符需要多调用
    一次即使用两次 cin.getline()
    cin.getline(cha, 50);                    //或者这之前使用 cin.ignore();
}

```

```

string str = string(cha);           //将字符数组转成字符串
int len = (int)str.length();       //表达式长度
int i = 0;

do {
    if (str[i] == '=' || str[i] == ' ') {           //当为空格或者 '=' 的时候，就直接跳过
        i++;
    }
    else if ( str[i] < 48 || str[i] > 57) {           //当不再 0-9 之间，就直接保存
        string temp = "";
        temp += str[i];
        this->infix_Expression.push_back(temp);
        i++;
    }
    else {
        string str_temp = "";
        while (i < len && str[i] >= 48 && str[i] <= 57) {    //当在 0-9 之间就拼接成原来的数字，然后再保存
            str_temp += str[i];
            i++;
        }
        this->infix_Expression.push_back(str_temp);
    }
} while (i < len);

this->to_Prefix();           //转换成前缀表达式
this->to_Postfix();          //转换成后缀表达式

//更新状态
this->is_Input = true;

cout << "=====" << endl;
cout << "表达式输入成功! :)" << endl;
cout << endl;

system("pause");
system("cls");
}

//转换成后缀表达式
void Caculate::to_Postfix() {
    //定义符号栈
    stack<string> sta;

```

```

    for (vector<string>::iterator it = this->infix_Expression.begin(); it != this->infix_Expression.end();
it++) {
        string str = *it;
        if (str >= "0" && str <= "9") {           //若是遇到数字就直接保存
            this->postfix_Expression.push_back(str);
        }
        else if (str == "(") {                     //遇到"(" 就进栈
            sta.push(str);
        }
        else if (str == ")") {                     //遇到")" 就去匹配此前的 "(",栈顶元素依次出栈, 并保存
            while (sta.top() != "(") {
                this->postfix_Expression.push_back(sta.top());
                sta.pop();
            }
            sta.pop();    //弹出栈顶的 ")"
        }
        else {
            while (sta.size() != 0 && get_Value(sta.top()) >= get_Value(str)) {
                this->postfix_Expression.push_back(sta.top());
                sta.pop();
            }
            sta.push(str);
        }
    }

    //如果栈中最后还有元素, 一并输出保存
    while (sta.size() != 0) {
        this->postfix_Expression.push_back(sta.top());
        sta.pop();
    }
}

```

//转换成前缀表达式

```

void Caculate::to_Prefix() {
    //定义符号栈
    stack<string> sta;
    vector<string> vec = this->infix_Expression;           //定义一个 vec 容器来存储中缀表达式, 方便
接下来的操作
    reverse(vec.begin(), vec.end());
    for (vector<string>::iterator it = vec.begin(); it != vec.end(); it++) {
        string str = *it;
        if (str >= "0" && str <= "9") {                 //若是遇到数字就直接保存
            this->prefix_Expression.push_back(str);
        }
    }
}

```

```

    }
    else if (str == ")") {          //遇到"(" 就进栈
        sta.push(str);
    }
    else if (str == "(") {
        while (sta.size() != 0 && sta.top() != ")") {
            this->prefix_Expression.push_back(sta.top());
            sta.pop();
        }
        sta.pop();    //将 ")"弹出符号栈
    }
    else {
        //如果当前栈顶元素为右括号 '}', 直接将操作符放入符号栈中
        if (!sta.empty() && sta.top() == ")") {
            sta.push(str);
        }
        else {
            //果当前栈顶元素的优先级大于操作数的优先级, 则将栈顶元素移除, 再次和判断
            //移除后栈的栈顶元素比较优先级大小,
            //直到当前栈顶元素小于或等于操作数优先级, 将操作符放入符号栈中
            while (!sta.empty() && get_Value(sta.top()) > get_Value(str)) {
                this->prefix_Expression.push_back(sta.top());
                sta.pop();
            }
            sta.push(str);
        }
    }
}

while (!sta.empty()) {            //最后当栈不为空的时候, 将栈中的元素依次移出并保存
    this->prefix_Expression.push_back(sta.top());
    sta.pop();
}
reverse(this->prefix_Expression.begin(), this->prefix_Expression.end());
}

//中缀表达式求值
void Caculate::caculate_Infix() {
    //定义两个栈, 一个运算符栈, 一个操作数栈
    stack<string> operator_Stack;
    stack<int> number_Stack;

    for (vector<string>::iterator it = this->infix_Expression.begin(); it != this->infix_Expression.end();
    it++) {

```

```

string str = *it;
stringstream ss;

int num = 0;
if (str[0] >= '0' && str[0] <= '9') { //如果第一个字符为数字字符，则这个字符串是数字，用
istringstream 转成数字
    istringstream istr(str);           //这样就可以实现多位整数的运算
    istr >> num;
    number_Stack.push(num);
}

else if (str == "(") {
    operator_Stack.push(str);
}
else if (str == ")") {
    while (operator_Stack.top() != "(") {
        int a = number_Stack.top();
        number_Stack.pop();
        int b = number_Stack.top();
        number_Stack.pop();
        int result = this->get_result(b, operator_Stack.top(), a);
        number_Stack.push(result);
        operator_Stack.pop();
    }
    operator_Stack.pop();           //弹出 "("
}
else {
    if (operator_Stack.size() == 0 || operator_Stack.top() == "(") {
        operator_Stack.push(str);
    }
    else {
        while (operator_Stack.size() != 0 && get_Value(operator_Stack.top()) >=
get_Value(str)) {
            int a = number_Stack.top();
            number_Stack.pop();
            int b = number_Stack.top();
            number_Stack.pop();
            int result = this->get_result(b, operator_Stack.top(), a);
            number_Stack.push(result);
            operator_Stack.pop();
        }
        operator_Stack.push(str);
    }
}
}

```

```

    }

    while (operator_Stack.size() != 0) {
        int a = number_Stack.top();
        number_Stack.pop();
        int b = number_Stack.top();
        number_Stack.pop();
        int result = this->get_result(b, operator_Stack.top(), a);
        number_Stack.push(result);
        operator_Stack.pop();
    }

    if (this->is_Input) {
        cout << "*****中缀表达式*****" << endl;
        this->show_Vector(this->infix_Expression);
    }
    else {
        cout << "Oops! 请先输入表达式！" << endl;
    }
    cout << "运算结果为： " << number_Stack.top() << endl;
    number_Stack.pop();

    system("pause");
    system("cls");
}

//前缀表达式求值
void Caculate::caculate_Prefix() {
    stack<int> sta;
    vector<string> vec = this->prefix_Expression;
    reverse(vec.begin(), vec.end());
    for (vector<string>::iterator it = vec.begin(); it != vec.end(); it++) {
        string str = *it;

        int num = 0;
        if (str[0] >= '0' && str[0] <= '9') { //如果第一个字符为数字字符，则这个字符串是数字，用
istringsteam 转成数字
            istringstream istr(str);           //这样就可以实现多位整数的运算
            istr >> num;
            sta.push(num);
        }

        else if(sta.size() !=0) {
            //注意与后缀表达式操作数顺序的区别

```



```

        int a = sta.top();
        sta.pop();
        int b = sta.top();
        sta.pop();
        int result = this->get_result(a, str, b);
        sta.push(result);
    }
}

if (this->is_Input) {
    cout << "*****前缀表达式*****" << endl;
    this->show_Vector(this->prefix_Expression);
}
else {
    cout << "Oops! 请先输入表达式！" << endl;
}
cout << "运算结果为： " << sta.top() << endl;
sta.pop();

system("pause");
system("cls");
}

//后缀表达式求值
void Caculate::caculate_Postfix() {
    stack<int> sta;

    for (vector<string>::iterator it = this->postfix_Expression.begin(); it != this->postfix_Expression.end();
it++) {
        string str = *it;
        stringstream ss;

        int num = 0;
        if (str[0] >= '0' && str[0] <= '9') { //如果第一个字符为数字字符，则这个字符串是数字，用
istream 转成数字
            istringstream istr(str); //这样就可以实现多位整数的运算
            istr >> num;
            sta.push(num);
        }
        else {
            int a = sta.top();
            sta.pop();
            int b = sta.top();
            sta.pop();
            int result = this->get_result(b, str, a);

```

```

        sta.push(result);
    }
}
if (this->is_Input) {
    cout << "*****后缀表达式*****" << endl;
    this->show_Vector(this->postfix_Expression);
}
else {
    cout << "Oops! 请先输入表达式！" << endl;
}
cout << "运算结果为： " << sta.top() << endl;
sta.pop();

system("pause");
system("cls");

}

//遍历展示 vector 容器中元素
void Caculate::show_Vector(vector<string> expression) {
    for (vector<string>::iterator it = expression.begin(); it != expression.end(); it++) {
        cout << (string)(*it) << " ";
    }
    cout << endl;
}

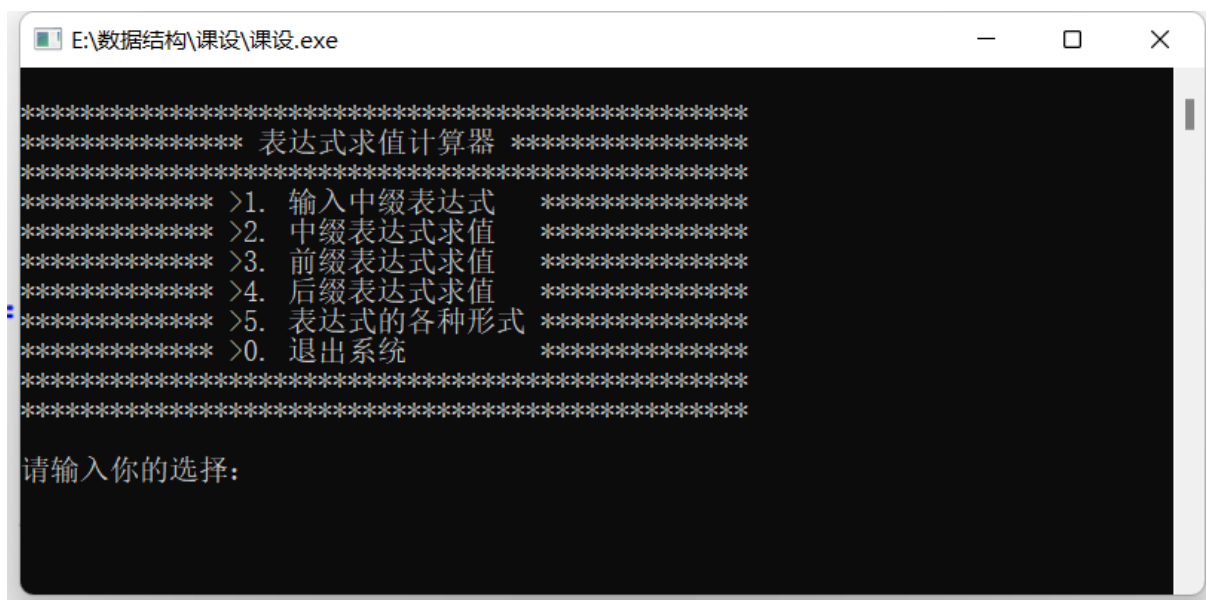
//展示表达式各种形式
void Caculate::show_Expression() {
    if (this->is_Input) {
        cout << "*****前缀表达式*****" << endl;
        this->show_Vector(this->prefix_Expression);
        cout << "*****中缀表达式*****" << endl;
        this->show_Vector(this->infix_Expression);
        cout << "*****后缀表达式*****" << endl;
        this->show_Vector(this->postfix_Expression);
    }
    else {
        cout << "Oops! 请先输入表达式！" << endl;
    }

    system("pause");
    system("cls");
}

```

五、运行结果

1.主界面

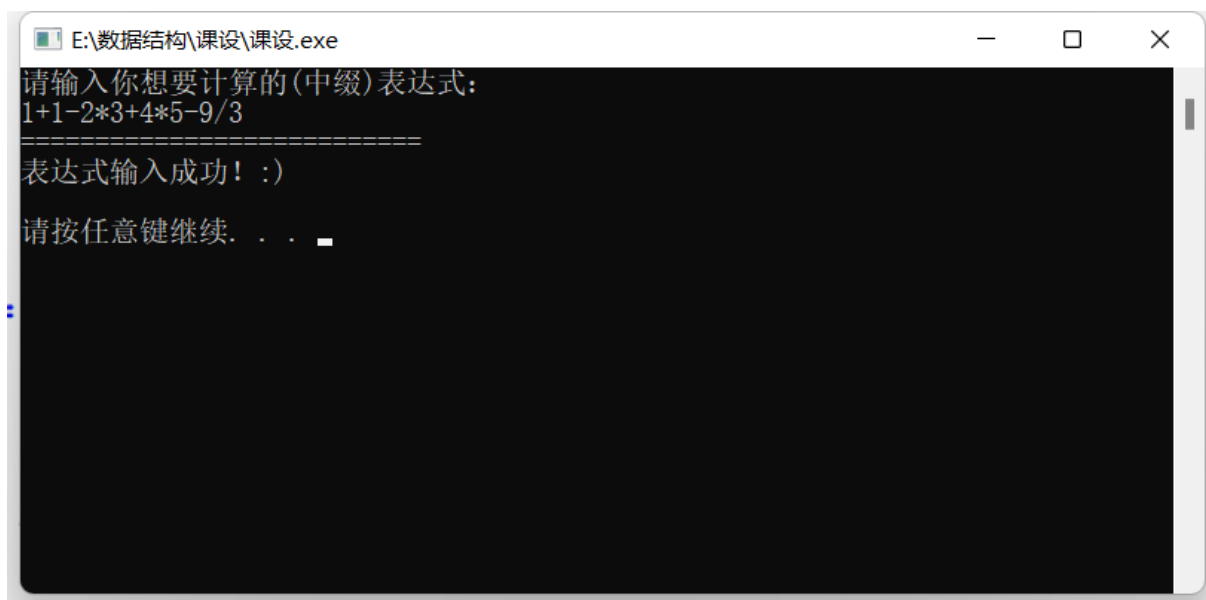


```
E:\数据结构\课设\课设.exe

*****
***** 表达式求值计算器 *****
*****
>1. 输入中缀表达式 *****
>2. 中缀表达式求值 *****
>3. 前缀表达式求值 *****
>4. 后缀表达式求值 *****
>5. 表达式的各种形式 *****
>0. 退出系统 *****
*****

请输入你的选择:
```

2.输入中缀表达式



```
E:\数据结构\课设\课设.exe

请输入你想要计算的(中缀)表达式:
1+1-2*3+4*5-9/3
=====
表达式输入成功! :)

请按任意键继续. . .
```

3.中缀表达式求值

```
E:\数据结构\课设\课设.exe

***** 表达式求值计算器 *****
*****
>1. 输入中缀表达式 *****
>2. 中缀表达式求值 *****
>3. 前缀表达式求值 *****
>4. 后缀表达式求值 *****
>5. 表达式的各种形式 *****
>0. 退出系统 *****

请输入你的选择: 2
*****中缀表达式*****
1 + 1 - 2 * 3 + 4 * 5 - 9 / 3
运算结果为: 13
请按任意键继续. . .
```

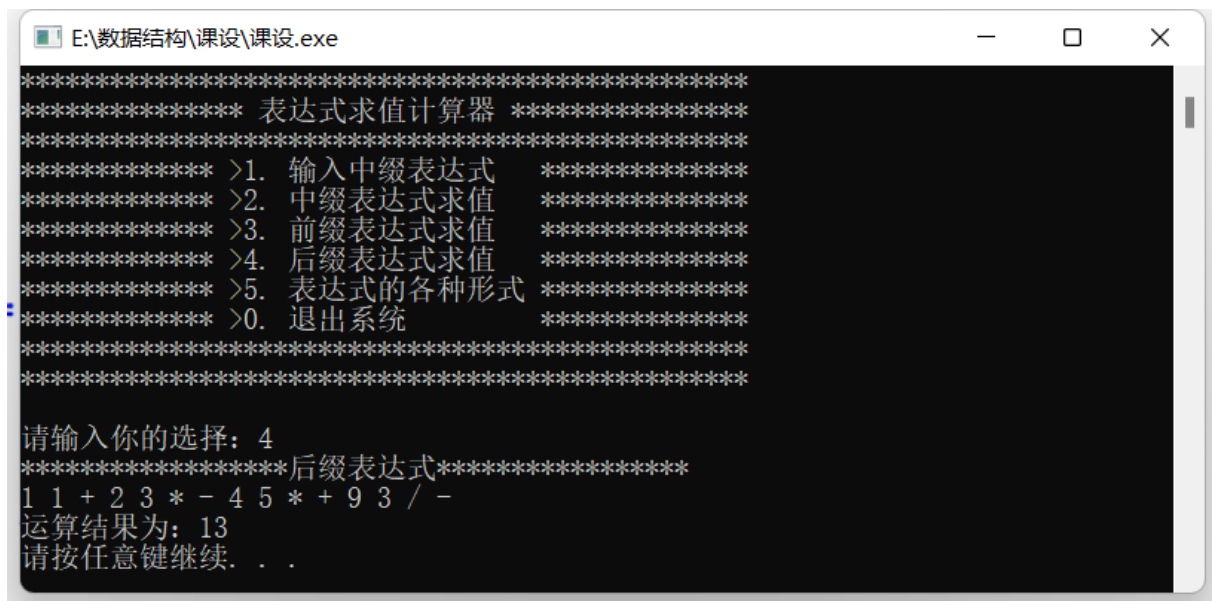
4. 前缀表达式求值

```
E:\数据结构\课设\课设.exe

***** 表达式求值计算器 *****
*****
>1. 输入中缀表达式 *****
>2. 中缀表达式求值 *****
>3. 前缀表达式求值 *****
>4. 后缀表达式求值 *****
>5. 表达式的各种形式 *****
>0. 退出系统 *****

请输入你的选择: 3
*****前缀表达式*****
- + - + 1 1 * 2 3 * 4 5 / 9 3
运算结果为: 13
请按任意键继续. . .
```

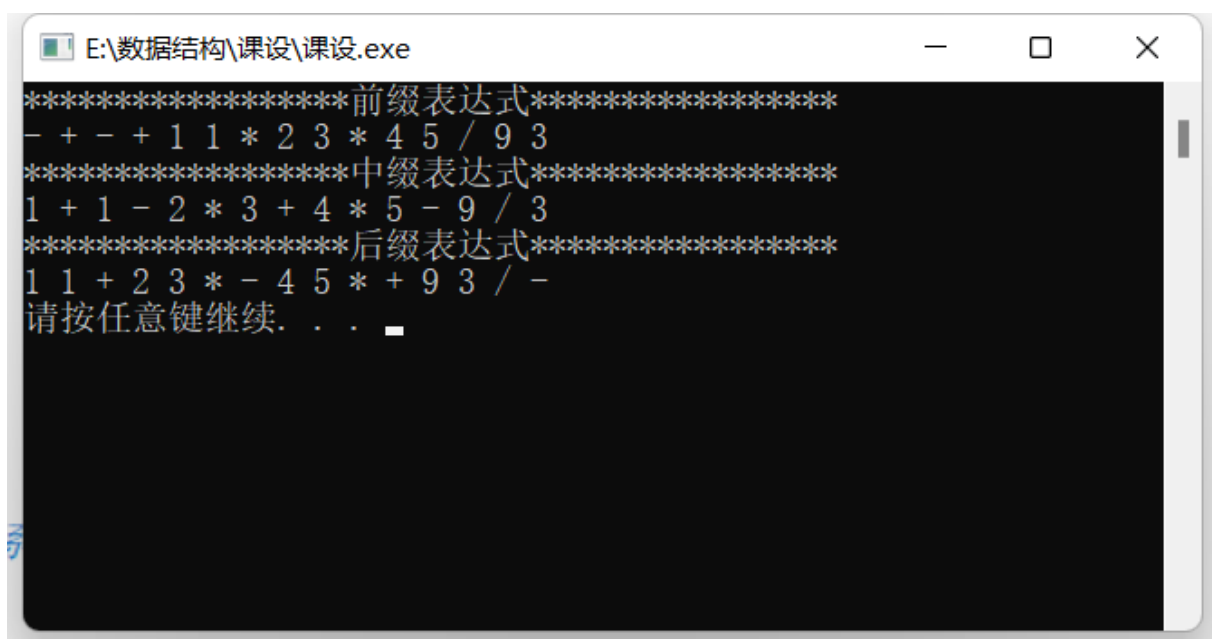
5. 后缀表达式求值



```
*****
***** 表达式求值计算器 *****
*****
***** >1. 输入中缀表达式 *****
***** >2. 中缀表达式求值 *****
***** >3. 前缀表达式求值 *****
***** >4. 后缀表达式求值 *****
***** >5. 表达式的各种形式 *****
***** >0. 退出系统 *****
*****
*****

请输入你的选择: 4
*****后缀表达式*****
1 1 + 2 3 * - 4 5 * + 9 3 / -
运算结果为: 13
请按任意键继续. . .
```

6. 表达式的各种形式



```
*****前缀表达式*****
- + - + 1 1 * 2 3 * 4 5 / 9 3
*****中缀表达式*****
1 + 1 - 2 * 3 + 4 * 5 - 9 / 3
*****后缀表达式*****
1 1 + 2 3 * - 4 5 * + 9 3 / -
请按任意键继续. . .
```

六、设计体会

本次数据结构表达式转换的课设设计让我更深入地学习了数据结构和算法，并锻炼了我的编程能力和解决问题的能力。通过这个设计过程，我对数据结构的应用有了更深入的理解，并学会了如何将理论知识应用到实际的程序设计中。同时，我也体会到了设计和调试一个完整程序的挑战和乐趣。

七、参考文献

- 1.数据结构（C 语言版）.严薇敏 吴伟民编著
2. [第 08 周 03--5.5 遍历二叉树和线索二叉树 5--二叉树的遍历算法--中序非递归算法 哔哩哔哩 bilibili](#)

八、教师评语

--